

DexVision Control Strategy Decision

Date: September 6, 2026

Purpose: Record why the project retired live hand-pose control and what replaces

it in the active roadmap.

What the project is

DexVision / Hand2Bot is a simulator-first robot-learning project built around a Shadow Hand in MuJoCo. It creates replayable manipulation trajectories, trains compact policies, and exposes qualified skills through a typed interface. Later, a language model may compose those qualified skills into work orders such as workspace clearing, inspection-station operation, and workspace setup. Language never controls joints directly.

The first two levels also produced an OpenCV/MediaPipe hand-tracking prototype and an immutable dataset derived from it. That work remains reproducible and useful as historical engineering evidence, but it is no longer part of the active manipulation-data or control architecture.

What has been completed

- Levels 1 and 2 established camera tracking, retargeting, recording, replay, quality checks, task metrics, and a versioned dataset release.
- Level 3 established reproducible behavior-cloning, checkpoint selection, and closed-loop evaluation, but none of its narrow-data policies qualified.
- Level 4.3 built and replay-qualified deterministic experts for reach, button press, constrained push, grasp-and-lift, and complete pick/place.
- Small state-only button and push pilots passed on held-out resets using a deterministic nominal controller plus a compact learned delta/residual.
- Level 4.4 collected 60 required scripted reach, push, and button episodes across all frozen core cells. Level 4.5 complete pick/place collection is next.

Why live hand control was retired

The Level 4 pilot corrected several real implementation problems, including the wrong hand anchor, an unsuitable orientation mapping, a simulation/control-rate mismatch, unclear target cues, and unsafe low transit paths. Even after those fixes, live monocular control was only usable for slow free-space reach. It did not reliably establish or maintain the constrained contacts required for button pressing, pushing, grasping, or placement.

Pilot action	Result	Decision
Reach	Eventually reached the cue after substantial control redesign	Scripted expert is still more repeatable and becomes the required source
Pick/place	Failed to grasp and disturbed a neighboring object	Retire live control
Push	Did not move the selected object toward its target	Retire live control
Button	Never reached a viable contact pose	Retire live control

Collecting more of the same attempts would increase effort without creating an expert dataset. A larger model, action chunking, or an LLM would not repair a bad demonstration source.

Active architecture

The remaining project uses this order:

1. Start from a typed task and simulator world state.
2. Generate motion with a deterministic task expert and safety checks.
3. Record requested, commanded, and applied actions.
4. Replay, recompute, and audit the dataset.
5. Train a compact state-grounded policy.
6. Add rendered perception grounding and the supervised skill executor.
7. Add language-guided composition only after qualification.

Required nominal trajectories come from deterministic scripted experts. Required correction examples are deterministic scripted continuations linked to retained scripted failures. Historical human-controlled attempts remain locally auditable with their original provenance but are excluded from the active dataset, release, training, correction, and evaluation path.

Near-term plan

1. Complete Level 4.5 scripted pick/place collection and replay checks.
2. Generate the frozen scripted failure and correction cells in Level 4.6.
3. Export aligned rendered visual supervision in Level 4.7.
4. Audit, package, checksum, and release the immutable Level 4 dataset.
5. In Level 5, train the smallest justified state-grounded policies first, then qualify rendered-perception variants and the supervised skill executor.
6. Add language-guided planning only after the skill registry qualifies.

Firm boundary

No remaining checkpoint should require a webcam, hand tracker, human-operated demonstration, or human corrective intervention. Manual MuJoCo viewer replay is still useful for verification because it observes a trajectory rather than controlling it.